```
$ octave -qf
octave:1> p = [0 0 5 3 -9 4 6];
octave:2> polyout(p, "x");
0*x^6 + 0*x^5 + 5*x^4 + 3*x^3 - 9*x^2 + 4*x^1 + 6
octave:3> polyreduce(p)
ans =

   5   3  -9   4   6

octave:4> polyout(polyreduce(p), "x");
5*x^4 + 3*x^3 - 9*x^2 + 4*x^1 + 6
octave:5>
```

Polynomial arithmetic

As Octave represents polynomials as vectors, their addition and subtraction are vector addition and subtraction, respectively. However, the polynomial vectors may be of different lengths. Hence, they need to be made the same length before addition or subtraction. So, let's write functions to do the complete operations:

```
function [ q1 q2 ] = equalize(p1, p2)
# Assuming p1 & p2 to be row vectors
    m = max(length(p1), length(p2));
    q1 = [ zeros(1, m - length(p1)) p1 ];
    q2 = [ zeros(1, m - length(p2)) p2 ];
endfunction

function p = polyadd(p1, p2)
    [ q1 q2 ] = equalize(p1, p2);
    p = polyreduce(q1 + q2);
endfunction

function p = polysub(p1, p2)
    [ q1 q2 ] = equalize(p1, p2);
    p = polyreduce(q1 - q2);
endfunction
```

Assuming the above code is put in the file *polynomials.oct*, the same can be used as follows:

```
$ octave -qf
octave:1> source("polynomials.oct");
octave:2> polyout(p1 = [ 1 2 1 ], "x");
1*x^2 + 2*x^1 + 1
octave:3> polyout(p2 = [ 1 -1 ], "x");
1*x^1 - 1
octave:4> polyout(polyadd(p1, p2), "x");
1*x^2 + 3*x^1 + 0
octave:5> polyout(polysub(p1, p2), "x");
1*x^2 + 1*x^1 + 2
octave:6>
```

Interestingly, Octave already has the functions for multiplication and division of polynomials, namely, *conv()* and *deconv()*, respectively. Here's a demonstration:

```
$ octave -qf
octave:1> polyout(p1 = [ 1 2 1 ], "x");
1*x^2 + 2*x^1 + 1
octave:2> polyout(p2 = [ 1 -1 ], "x");
1*x^1 - 1
octave:3> polyout(conv(p1, p2), "x");
1*x^3 + 1*x^2 - 1*x^1 - 1
octave:4> polyout(deconv(p1, p2), "x");
1*x^1 + 3
octave:5> [ q r ] = deconv(p1, p2)
q =

   1   3

r =

   0   0   4

octave:6>
```

Polynomial differentiation and integration

Octave also provides functions for differentiation and integration of polynomials, namely, *polyder()* and *polyint()*. Here is an example of differentiation and definite integral using the same:

```
$ octave -qf
octave:1> polyout(p = [ 1 2 1 ], "x");
1*x^2 + 2*x^1 + 1
octave:2> polyout(polyder(p), "x");
2*x^1 + 2
octave:3> polyout(polyint(p), "x");
0.33333*x^3 + 1*x^2 + 1*x^1 + 0
octave:4> q = polyint(p);
octave:5> polyval(q, 3) - polyval(q, 0) # Definite integral of
p from 0 to 3
ans =  21
octave:6>
```

What's next?

Given a set of data points, a common requirement in the fields of physics, statistics and many others is to fit a polynomial to it. Going further, we'll explore the power of Octave to do just that. END

By: Anil Kumar Pugalia

The author is a gold medallist from the Indian Institute of Science, and has a passion for mathematics and knowledge sharing. He is a hobbyist in open source software and hardware. He experiments with Linux and embedded systems, and shares his knowledge through weekend workshops *(http://sysplay.in)*. Reach him at *email@sarika-pugs.com*.